

Training Run v2: Multi-Trace Augmentation, Throughput Debugging, and Reward A

Pulak Mehrotra

March 28–30, 2026

Abstract

This report documents the first end-to-end training runs of the Octopus CXL memory-pooling RL agent using the full 10-trace augmented dataset (`--multi-trace`, `--augmentation`, $N_{\text{envs}} = 32$). Three system-level bugs were identified and fixed: a copy-on-write OOM during `SubprocVecEnv` fork, a pickle import-lock deadlock in parallel trace loading, and a `KeyError: 192` crash in `PoolingSavingsCallback` caused by building `mhd.to_hosts` from the base pod matrix rather than the full block-diagonal `expanded_M`. Throughput profiling identified eval callbacks (not gradient updates or environment steps) as the true bottleneck, and a `train_freq=8`, `eval_freq=100000` configuration restores $\approx 2,400$ fps. The original reward (`current`) showed near-zero policy improvement; Reward A with a 50-dim observation space is the active variant under test.

1 Run Configuration

```
1 python scripts/train_rl.py \  
2   --run-id aug_all_traces \  
3   --augmentation \  
4   --multi-trace \  
5   --aug-traces AMS20PrdApp19-tround BLAPrdApp19-troundgrt5m \  
6                 BN9PrdApp18-troundgrt5m DSM08PrdApp05-troundgrt5m \  
7                 DUB24PrdApp09-troundgrt5m LON23PrdApp01-troundgrt5m \  
8                 LVL01PrdApp05-troundgrt5m SG2PrdApp35-troundgrt5m \  
9                 SYD21PrdApp07-troundgrt5m YT021PrdApp05-troundgrt5m \  
10  --wandb --n-envs 32 --total-timesteps 500000
```

Key parameters: reward variant = `current`, $\lambda_{\text{var}} = 0.5$, batch size = 256, $\gamma = 0.999$, `train_freq=1`, `gradient_steps=1`, network = [256, 256] MLP, eval trace = `LON23PrdApp01-troundgrt5m`.

2 System Fixes Required

2.1 Copy-on-Write OOM During Worker Fork

The original run crashed silently after loading all 10 traces. The OOM killer terminated the process with no Python traceback.

Root cause. Python’s reference counting increments `ob_refcnt` on every read of a Python object, which marks the memory page as dirty and triggers a copy-on-write copy in each forked subprocess. With 10 traces loaded as Python object graphs (~ 53 GB in memory) and 32 workers each copying ~ 7 GB of CoW pages, peak memory reached ~ 277 GB on a 192 GB machine.

Fix. Trace data is now converted to `TraceArrays` (7 numpy arrays: `vm_start`, `vm_end`, `vm_mem`, `node_ids`, `node_dram`, `node_offsets`, `vm_ptrs`) via `octopus.data.to_arrays()` before the `SubprocVecEnv` fork. Reading numpy C-heap buffers does not touch Python refcounts, so no CoW pages are generated. Total pool footprint dropped from ~ 53 GB to ~ 300 MB.

2.2 Pickle Import-Lock Deadlock in Parallel Loading

A `ThreadPoolExecutor` was used to load all 10 traces simultaneously. This deadlocked because `pickle.Unpickler.find_class` calls `__import__`, which acquires Python’s per-module import lock. Ten threads all calling `__import__` simultaneously caused a classic import-lock deadlock.

Fix. Reverted to sequential loading. Startup time is ≈ 10.5 min, dominated by I/O-bound pickle deserialization and the `to_arrays()` conversion.

3 Startup Time Breakdown

Phase	Wall time	Cumulative
W&B init	0:00 – 0:09:30	9 min 30 s
Training trace load + topology	0:09:30 – 0:10:00	30 s
10-trace pool load (sequential)	0:10:00 – 0:18:31	8 min 31 s
SubprocVecEnv fork (32 workers)	0:18:31 – 0:20:34	2 min 03 s
Eval trace load	0:20:34 – 0:20:56	22 s
Total to first step		≈ 20 min 56 s

Table 1: Startup phases. Excluding W&B, data loading accounts for ≈ 11 min 26 s.

The 10-trace pool load scales linearly with trace size. The largest traces (SG2: 1.7 GB, BLA: 1.6 GB, BN9: 1.5 GB on disk) dominate, taking 60–80 s each. `to_arrays()` conversion adds a fixed per-trace overhead of ≈ 5 –15 s (tight Python loop over VM objects to extract int64 timestamps and float32 memory values).

4 Training Throughput

4.1 Observed vs. Expected

Checkpoint	time_elapsed (s)	total_timesteps	fps
115,296	1,431	115,296	80
206,528	2,837	206,528	72
215,488	2,850	215,488	75
Average			75

Table 2: Observed training throughput. $\text{fps} = \text{total_timesteps} / \text{time_elapsed}$.

The benchmark from training-v1 with $N = 32$ `SubprocVecEnv` was **1,221 fps**. The current run achieves **75 fps** — a **16 $\times$ regression**.

4.2 Projected Wall Times

Total timesteps	Training time	+ Startup (≈ 11 min)
500k	1.85 hr	≈ 2.0 hr
1M	3.7 hr	≈ 3.9 hr
2M	7.4 hr	≈ 7.6 hr
5M	18.5 hr	≈ 18.7 hr

Table 3: Projected run times at 75 fps. Overnight budget ≈ 8 hr $\Rightarrow$ 2M timesteps feasible.

4.3 Root Cause: Eval Callbacks, Not Gradient Updates

Two profiling experiments identified the true bottleneck.

Experiment 1 — **environment-only profiling** (no gradient updates, `scripts/profile_throughput.py`):

Configuration	fps	vs. baseline
Single trace, no aug, no multi-trace	2,806	$1.0\times$
+ Multi-trace only	2,611	$0.93\times$
+ Augmentation only	2,760	$0.98\times$
+ Both (current training config)	2,565	$0.91\times$

Table 4: Environment throughput without gradient updates. Augmentation and multi-trace have negligible impact ($<10\%$). All four run $\gg 75$ fps — the env is not the bottleneck.

Experiment 2 — **train_freq sweep with gradient updates** (`scripts/smoke_train_freq.py`), $N = 32$ envs, 3k steps, no callbacks:

train_freq	fps	vs. tf=1	grad updates	grad mean	collect mean
1	1,031	$1.0\times$	79	24 ms	5 ms
4	1,790	$1.7\times$	21	24 ms	23 ms
8	2,454	$2.4\times$	11	19 ms	33 ms
16	2,715	$2.6\times$	6	17 ms	65 ms
32	2,484	$2.4\times$	3	24 ms	167 ms

Table 5: Each gradient update costs ≈ 20 – 25 ms. The system without callbacks reaches 1,000–2,700 fps depending on `train_freq`. `train_freq=8` is the sweet spot: $2.4\times$ speedup with $2\times$ more updates than `tf=16` for better sample efficiency.

Conclusion. The env+gradient system achieves $\approx 1,000$ fps at `train_freq=1` without callbacks. Real training achieves 75 fps. The gap — ≈ 266 seconds per 20k-step eval window — is entirely attributable to eval callbacks:

- **PoolingSavingsCallback** (dominant cost): runs `pooling_simulation` for 10 full inference episodes every `--eval_freq=20000` steps. Each inference episode is $\sim 7,000$ serial steps. Total: $\sim 70,000$ inference steps per eval window, amortised as ≈ 220 ms per training step.
- **EvalCallback**: 3 eval episodes $\times$ 7,000 steps $\approx 21,000$ inference steps. Amortised ≈ 65 ms per training step.

4.4 Are the Callbacks Necessary?

Callback	Affects training?	Cost	Verdict
EvalCallback	Yes — saves <code>best_model.zip</code>	Medium	Keep, raise freq
PoolingSavingsCallback	No — diagnostic only	High	Raise freq or drop
CheckpointCallback	No — periodic snapshots	Low	Keep
SACDiagnosticsCallback	No — logs Q/entropy	Negligible	Keep
WandbCallback	No — artifact upload	Low	Keep

Table 6: `PoolingSavingsCallback` does **not** save checkpoints or influence the gradient signal. It is purely diagnostic and accounts for $\sim 80\%$ of callback overhead.

`EvalCallback` is worth retaining because it is the only mechanism that saves the best policy during training — critical if the reward improves then regresses. However, both callbacks should run far less frequently.

4.5 Projected Wall Times After Fix

Recommended config: `--train-freq 8 --eval-freq 100000`.

Total timesteps	Estimated time	Notes
500k	~ 12 min	
1M	~ 24 min	
2M	~ 48 min	Overnight budget
5M	~ 2 hr	

Table 7: Projected at $\approx 2,400$ fps (tf=8, eval-freq=100k). Startup adds ≈ 11 min.

5 Learning Signal Analysis

5.1 Pooling Savings

The policy achieves $\approx 1.4\%$ **savings over greedy** at peak, but this improvement is **not monotonically increasing** and shows no upward trend after 60k steps. The savings are within one standard deviation of zero at most checkpoints, suggesting the policy has not found a meaningfully better strategy than the greedy baseline.

5.2 SAC Internals

Several internal SAC metrics signal problematic learning dynamics:

- **Entropy collapse:** `ent_coef` decays monotonically from 0.837 (20k steps) to 0.155 (160k steps). The policy is becoming deterministic, but without finding a better optimum.
- **Unchecked Q-value growth:** `q1_mean` rises from 12.5 to >70 over the run. This is consistent with the Q-function tracking cumulative negative reward without a bound, but at a rate suggesting value overestimation.

Timesteps	pooling_savings_mean	pooling_savings_std
20k	0.0048	0.0275
40k	0.0128	0.0164
60k	0.0156	0.0131
80k	0.0140	0.0128
100k	0.0139	0.0119
120k	0.0131	0.0148
140k	0.0139	0.0151
160k	0.0101	0.0132
180k	0.0068	0.0131
200k	0.0139	0.0090

Table 8: `pooling_savings_mean` = fraction by which RL outperforms the greedy baseline on the eval trace (LON23). Standard deviation across 10 eval episodes.

- **Negative training-env savings:** `env/pooling_savings` is negative throughout (-0.10 to -0.44), meaning the policy performs *worse* than greedy on the training traces — it only marginally outperforms greedy on the held-out eval trace.
- **Critic loss converging:** `critic_loss` drops from 0.074 to 0.004 , indicating the value function has converged, but to a poor policy.

5.3 Why Reward “current” Fails

The reward $R = -(\Delta\text{peak})/\text{fair_share} - \lambda \cdot \text{Var}(\mathbf{c}/\text{fair_share})$ has three structural problems:

1. **Sparse signal:** only the single worst MPD contributes to the peak-delta term. Most decisions receive near-zero gradient.
2. **Myopic:** no lookahead. The agent cannot anticipate departures or future arrivals, so it cannot balance load over time.
3. **Scale sensitivity:** the fair-share normalization depends on pod-level DRAM, which varies per episode under multi-trace augmentation ($5\text{--}7\times$ scale range). The reward magnitude is inconsistent across episodes, harming value function stability.

6 First Reward A Run: Crash and Fix

Armed with the throughput fixes (`train_freq=8`, `eval_freq=100000`), the first Reward A run (`--run-id aug_v2_rewardA`) was launched.

6.1 Run Configuration

```

1 python scripts/train_rl.py \
2   --run-id aug_v2_rewardA \
3   --reward-variant A \
4   --augmentation --multi-trace \
5   --aug-traces AMS20PrdApp19-tround BLAPrdApp19-troundgrt5m \
6   BN9PrdApp18-troundgrt5m DSM08PrdApp05-troundgrt5m \

```

```

7         DUB24PrdApp09-troundgrt5m LON23PrdApp01-troundgrt5m \
8         LVL01PrdApp05-troundgrt5m SG2PrdApp35-troundgrt5m \
9         SYD21PrdApp07-troundgrt5m YT021PrdApp05-troundgrt5m \
10    --n-envs 32 --total-timesteps 2000000 \
11    --train-freq 8 --eval-freq 100000

```

6.2 Early Signal at 100k Steps

The first eval completed successfully at 100k steps:

Metric	Value
pooling_ratio	1.05
pooling_savings	-0.053 (5.3% worse than greedy)
mean_reward	-25.1
episode_length	7,334
critic_loss	0.112
ent_coef	0.891
n_updates	387

Table 9: Eval metrics at 100k steps. High entropy (0.891) and elevated critic loss indicate the policy is still in early exploration. Negative savings at 100k is expected for Reward A — the 50-dim observation requires more experience before the value function stabilises.

6.3 Crash: KeyError: 192 in PoolingSavingsCallback

The run crashed immediately after the 100k-step eval (first checkpoint saved), during the next PoolingSavingsCallback invocation:

```

1 File "scripts/evaluate.py", line 344, in _rl_alloc_cb
2     P_j = float(sum(host_cx1_load[h] for h in mhd_to_hosts[mhd]))
3 KeyError: 192

```

Root cause. `expand_M_to_all_nodes` assembles a block-diagonal topology matrix where pod 0 occupies MPD columns 0–191, pod 1 occupies columns 192–383, and so on. The `mhd_to_hosts` dict was precomputed once from the base per-pod matrix M (192 entries, keys 0–191) and stored in the callback at startup. When the callback evaluated a pod sampled from block 1 or higher, `mhd_list` contained global column indices ≥ 192 , which were absent from `mhd_to_hosts`.

The first eval (100k steps) happened to sample only pod 0 in all 10 iterations (seed 99999–100008 all landed there), masking the bug. The second eval (200k steps) sampled at least one pod with index ≥ 1 , triggering the crash.

Fix. `PoolingSavingsCallback.run_eval()` now rebuilds `mhd_to_hosts` and `Q_j` from `expanded_M` inside each iteration, immediately after `expand_M_to_all_nodes` returns. The rebuilt structures cover all `num_pods` $\times$ 192 global MPD indices:

```

1 node_to_M, expanded_M = expand_M_to_all_nodes(self._M, pod_to_nodes)
2 if self._obs_variant != "current":
3     num_mhd_exp = len(expanded_M[0])
4     mhd_to_hosts_exp = {j: [] for j in range(num_mhd_exp)}
5     h_to_mhds = {}
6     for h, row in enumerate(expanded_M):
7         h_to_mhds[h] = [j for j, v in enumerate(row) if v]

```

```

8         for j in h_to_mhds[h]:
9             mhd_to_hosts_exp[j].append(h)
10    Q_j_exp = np.zeros(num_mhd_exp, dtype=np.float32)
11    for j in range(num_mhd_exp):
12        hosts = mhd_to_hosts_exp[j]
13        if hosts:
14            inv_deg = [1.0/len(h_to_mhds[h]) for h in hosts if h_to_mhds[h]]
15            if inv_deg:
16                Q_j_exp[j] = float(np.mean(inv_deg))

```

The pre-built `self.mhd_to_hosts` (base-pod scope) is no longer passed to `make_rl_alloc_cb` for obs variants other than `current`.

7 Next Steps

1. **Re-run Reward A with the KeyError fix** (in progress). Same config as Section 6 but with the corrected `_run_eval`. 2M timesteps at $\approx 2,400$ fps $\Rightarrow \approx 14$ min training, ≈ 8 min startup. Monitor `eval/pooling_savings_mean`: crossing zero means the policy beats greedy; $>1.5\%$ is a meaningful result.
2. **Convergence criterion.** There is no fixed step budget. Stop when `pooling_savings_mean` plateaus across 3–5 consecutive eval windows (each 100k steps). If it crosses zero and keeps rising, extend to 5M–10M steps.
3. **If Reward A converges cleanly**, ablate Reward B (which adds host-pressure P_j and scarcity Q_j to the observation) to determine whether the richer state space improves asymptotic savings.